# Lecture 4
# Multithreaded programming

Dr.Negm Shawky

EMAIL: negmshawky@gmail.com

# Outline

- Introduction
- Thread Applications
- Defining Threads
- Java Threads and States
- Architecture of Multithreaded servers
- Threads Synchronization
- Summary

# Basics

## Multithreaded programming



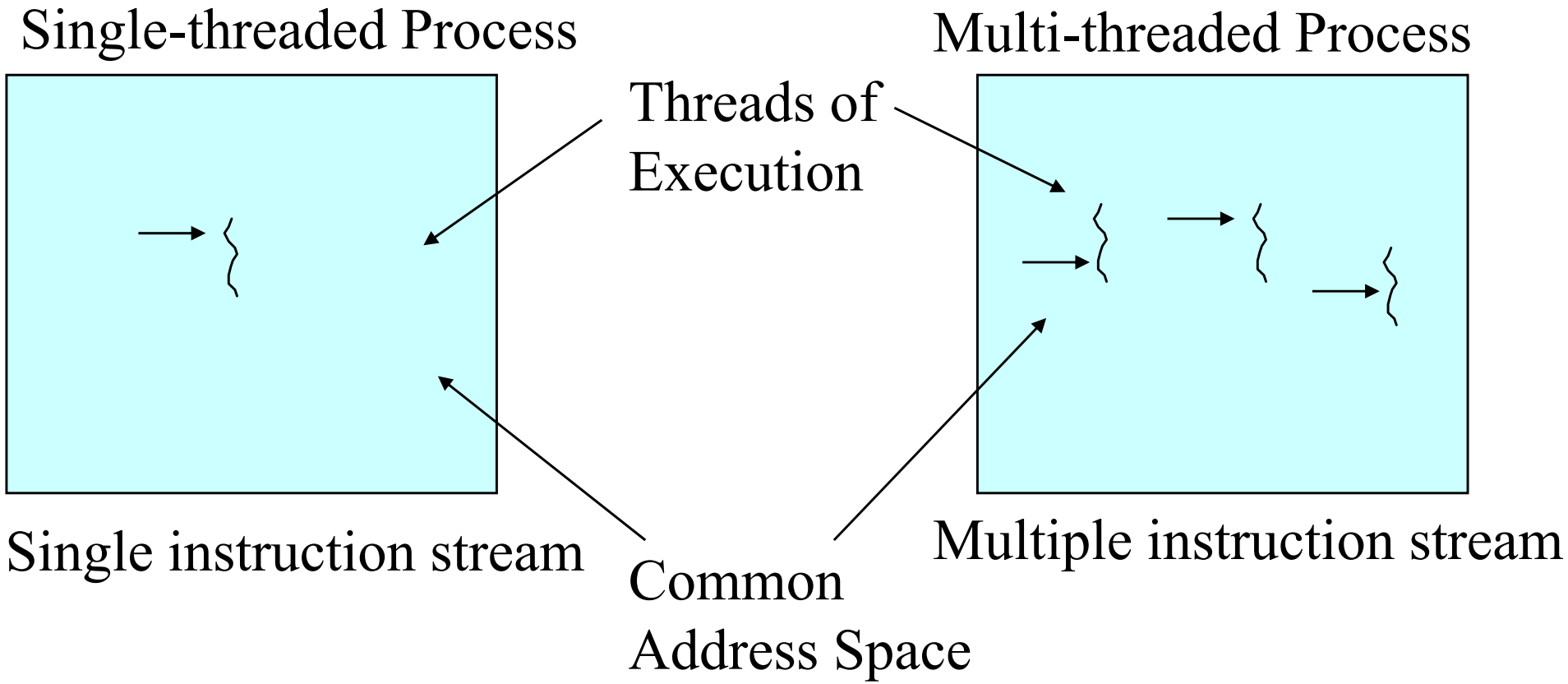
How you can concentrate in the lecture

# DEFINING THREADS

- To understand multithreading, the concepts *process* and *thread* must be understood.
- A *process* is a program in execution.
- A process may be divided into a number of independent units known as *threads*.
- A process is a collection of one or more threads and associated system resources.
- A process may have a number of threads in it.
- A thread may be assumed as a subset of a process.
- The difference between a process and a thread is shown in the following Fig.

# Single and Multithreaded Processes

**A process containing single and multiple threads**

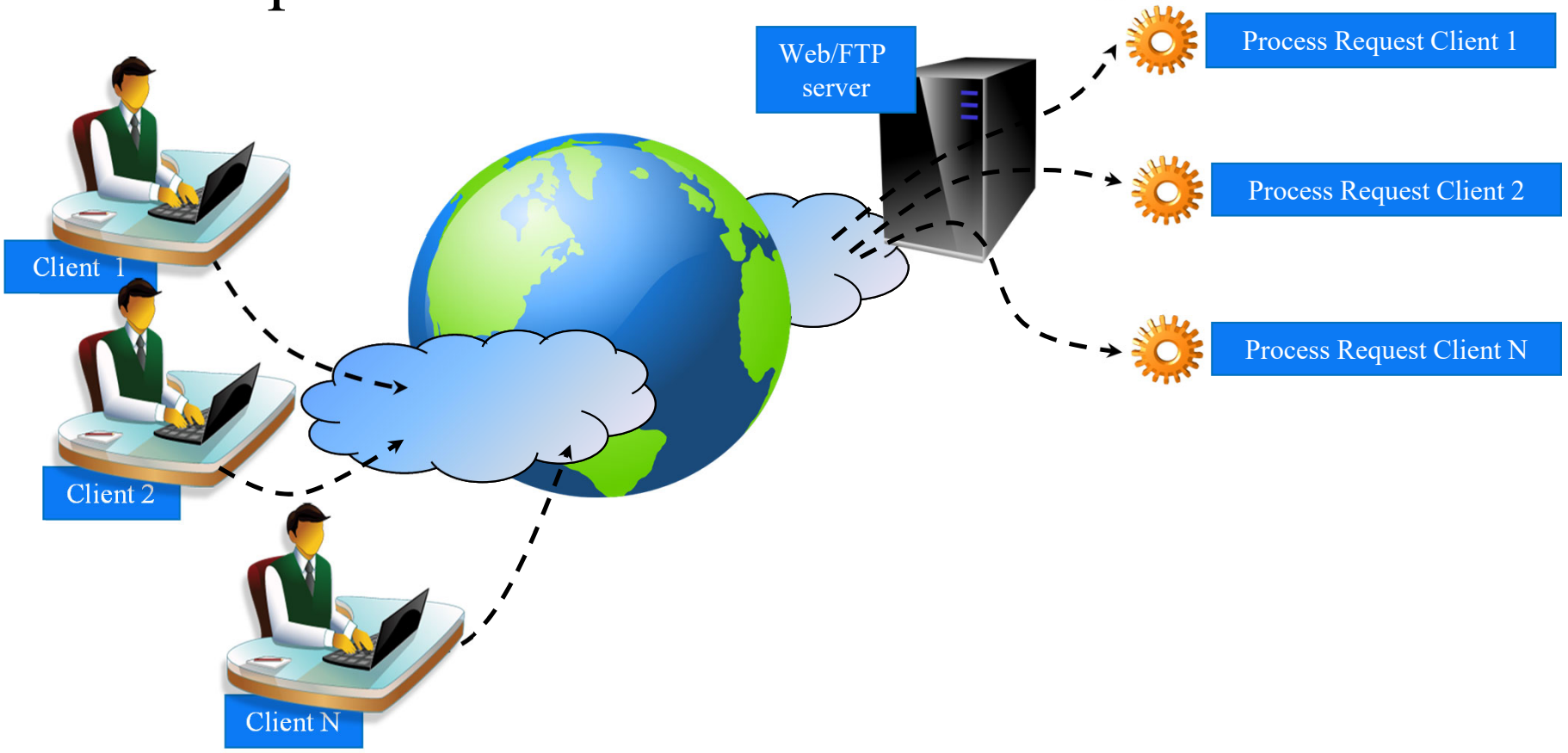


# DEFINING THREADS

- If two applications are run on a computer (MS Word, MS Access), two processes are created.
- Multitasking of two or more processes is known as *process-based multitasking*.
- Multitasking of two or more threads is known as *thread-based multitasking*.
- The concept of multithreading in a programming language refers to thread-based multitasking.
- Process-based multitasking is totally controlled by the operating system.
- But thread-based multitasking can be controlled by the programmer to some extent in a program.

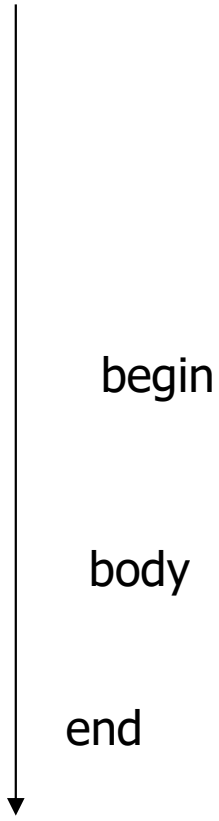
# Multithreaded Server: For Serving Multiple Clients Concurrently

- Modern Applications
  - Example: Multithreaded Web Server



# A single threaded program

```
class ABC
{
....
    public void main(..)
    {
    ...
    ..
    }
}
```



# Threads in java

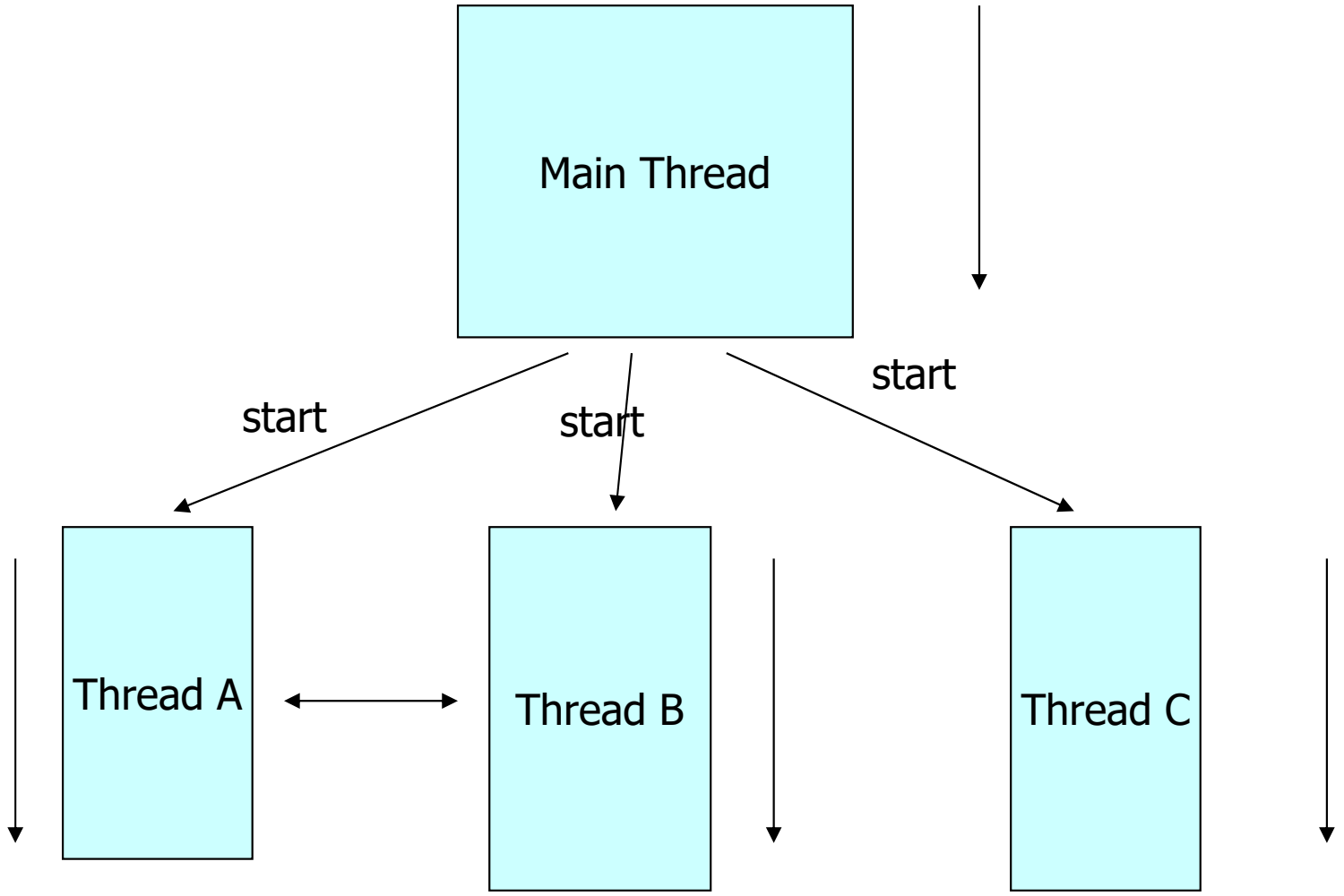
- A Thread is a piece of code that can run concurrently with other threads.
- Each thread is a statically ordered sequence of instructions.
- Programming a task having multiple threads of control – called Multithreaded Programming.

# Threads in java

- Applications are typically divided into processes during the design phase, and a master process explicitly produces subprocesses.
- A single process might contain multiple threads (see the following Fig.).
- All threads within a process share the same state and same memory space, and can communicate with each other directly, because they share the same variables.

# A Multithreaded Program

**A program with master thread and children threads**




Threads may switch or exchange data/results

# Threads in java

- Threads are objects in the Java language.
- They can be created by using three different mechanisms as illustrated in the following:

1. Create a class that extends the standard *Thread* class.
2. Create a class that implements the standard *Runnable* interface.
3. Creating a Thread Object with a Lambda Expression

# 1st method: Extending Thread class

- Create a class by extending Thread class and override run() method:

```
class MyThread extends Thread
{
     public void run()
    {
        // thread body of execution
    }
}
```

- Create a thread:

```
MyThread thr1 = new MyThread();
```

- Start Execution of threads:

```
thr1.start();
```

- Or you can Create and Execute in one statment:

```
new MyThread().start();
```

# Threads in java

- The run() method should be overridden and should contain the code that will be executed by the new thread.
- This method must be **public** with a **void** return type and should not take any arguments.

# An example

- /* ThreadEx1.java: A simple program creating and invoking a thread object by extending the standard Thread class. */

```
class MyThread extends Thread {
        public void run() {
                System.out.println(" this thread is running ... ");
        }
}

class ThreadEx1 {
        public static void main(String [] args  ) {
            MyThread t = new MyThread();
             t.start();
       }
}
```

# 2nd method: Threads by implementing Runnable interface

- Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{
   .....
   public void run()
   {
      // thread body of execution
   }
}
```

- Creating runnable Object:

```
MyThread myObject = new MyThread();
```

- Creating Thread Object:

```
Thread thr1 = new Thread( myObject );
```

- Start Execution:

```
thr1.start();
```

# An example

- /* ThreadEx2.java: A simple program creating and invoking a thread object by implementing Runnable interface. */

```
class MyThread implements Runnable  {
        public void run() {
                System.out.println(" this thread is running ... ");
        }
}

class ThreadEx2 {
        public static void main(String [] args  ) {
                Thread t = new Thread(new MyThread());
                 t.start();
        }
}
```

# An example

- The class MyThread implements standard Runnable interface and overrides the run() method that includes logic associated with the body of the thread (step 1).
- The objects created by the class MyThread are normal objects (unlike the first mechanism) (step 2).
- Therefore, we need to create a generic Thread object and pass MyThread object as a parameter to this generic Thread object (step 3).
- As a result of this association, threaded object is created.
- In order to execute this threaded object, we need to invoke its start() method which sets execution of the new thread (step 4)

# 3rd method: Threads with a lambda expression

## Method 3: Creating a Thread Object with a Lambda Expression (Second Slide)

In this approach, you directly create a Thread object and provide the task that the thread should execute.

```
Thread nodeA = new Thread(() -> {
for (int i = 1; i <= 3; i++) {
System.out.println("Node-A processing task " + i);
}
});

nodeA.start();
```

**How it works**

1. Create a Thread object.
2. Pass a task (Runnable) using a lambda expression () -> { }.
3. Call start() to execute the task concurrently.
4. **task** executed by the thread.

# 3rd method: Threads with a lambda expression

The lambda expression is simply a shorter form of implementing Runnable.

**Advantages**

- More flexible.
- Your class can still extend another class.
- Preferred in modern Java programming.
- Better separation between the **thread** and the **task** executed by the thread.

# A Program with Three Java Threads

- Write a program that creates 3 threads

# Three threads example

```
class A extends Thread
{
      public void run()
       {
           for(int i=1;i<=5;i++)
            {
                System.out.println("\t From ThreadA: i= "+i);
            }
             System.out.println("Exit from A");
       }
}
```

# Three threads example

```
class B extends Thread
{
      public void run()
       {
          for(int j=1;j<=5;j++)
           {
               System.out.println("\t From ThreadB: j= "+j);
           }
            System.out.println("Exit from B");
       }
}
```

# Three threads example

```
class C extends Thread
{
      public void run()
       {
          for(int k=1;k<=5;k++)
           {
               System.out.println("\t From ThreadC: k= "+k);
           }

            System.out.println("Exit from C");
       }
}
```

# Three threads example

```
class ThreadTest
{
        public static void main(String args[])
         {
                new A().start();
                new B().start();
                new C().start();
         }
}
```

# Run 1

- [raj@mundroo] threads [1:76] java ThreadTest
  From ThreadA: i= 1
  From ThreadA: i= 2
  From ThreadA: i= 3
  From ThreadA: i= 4
  From ThreadA: i= 5

Exit from A
  From ThreadC: k= 1
  From ThreadC: k= 2
  From ThreadC: k= 3
  From ThreadC: k= 4
  From ThreadC: k= 5

Exit from C
  From ThreadB: j= 1
  From ThreadB: j= 2
  From ThreadB: j= 3
  From ThreadB: j= 4
  From ThreadB: j= 5

Exit from B

# Run 2

- [raj@mundroo] threads [1:77] java ThreadTest
  From ThreadA: i= 1
  From ThreadA: i= 2
  From ThreadA: i= 3
  From ThreadA: i= 4
  From ThreadA: i= 5
  From ThreadC: k= 1
  From ThreadC: k= 2
  From ThreadC: k= 3
  From ThreadC: k= 4
  From ThreadC: k= 5
Exit from C
  From ThreadB: j= 1
  From ThreadB: j= 2
  From ThreadB: j= 3
  From ThreadB: j= 4
  From ThreadB: j= 5
Exit from B
Exit from A

# A JAVA PROGRAM WITH MULTIPLE THREADS

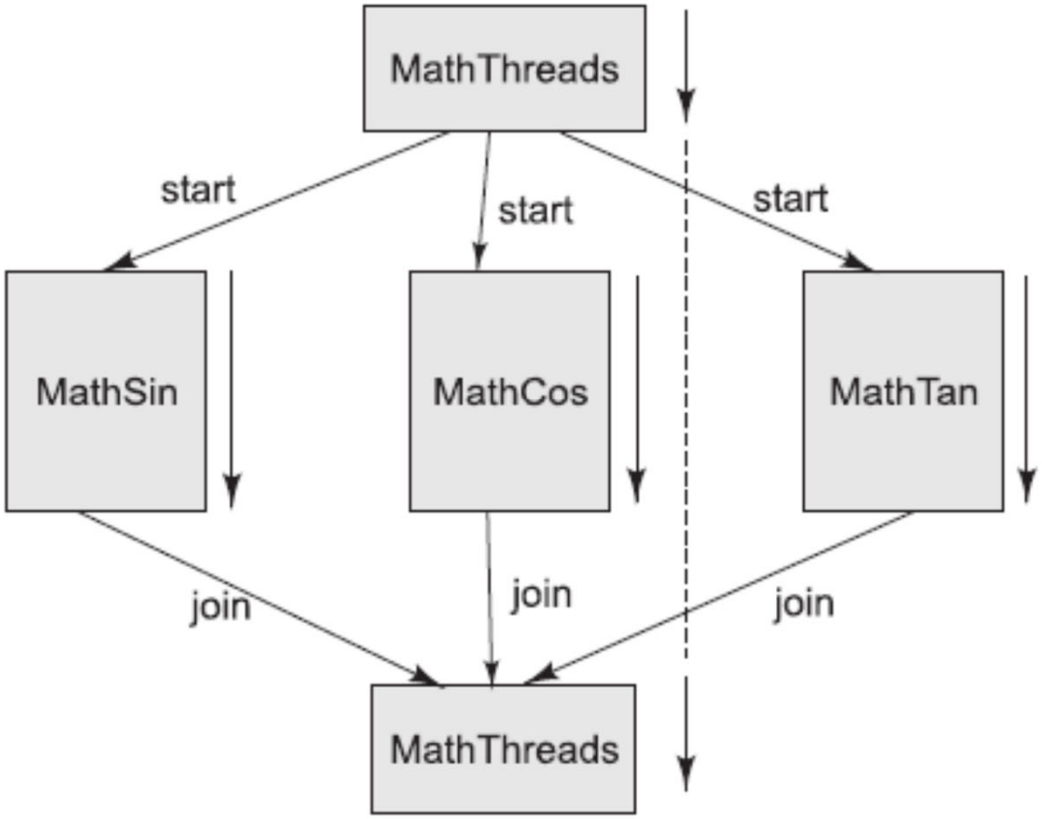
- To illustrate creation of multiple threads in a program performing concurrent operations, let us consider the processing of the following mathematical equation:

$$p = \sin(x) + \cos(y) + \tan(z)$$

- these trigonometric functions are independent operations without any dependencies between them, and these can be executed concurrently.
- After that their results can be combined to produce the final result as illustrated in the following Fig.

# A JAVA PROGRAM WITH MULTIPLE THREADS

- **Flow of control in a master and multiple workers threads application**



# A JAVA PROGRAM WITH MULTIPLE THREADS

- The main/default thread is called MathThreads, which acts like a master thread. It creates three worker threads (MathSin, MathCos, and MathTan) and assigns them to compute values for different data inputs.
- All three worker threads are concurrently executed on shared or dedicated CPUs depending on the type of machine.
- Although the master thread can continue its execution, in this case, it needs to make sure that all operations are completed before combining individual results.
- This is accomplished by waiting for each thread to complete by invoking join() method associated with each worker thread.
- A complete Java program illustrating this concept is given in the following Program.

# A JAVA PROGRAM WITH MULTIPLE THREADS

**Program**

```
/* MathThreads.java: A program with multiple threads performing concurrent operations. */
import java.lang.Math;
class MathSin extends Thread {
   public double deg;
   public double res;
   public MathSin(int degree) {
     deg = degree;
   }
   public void run() {
     System.out.println("Executing sin of "+deg);
     double Deg2Rad = Math.toRadians(deg);
     res = Math.sin(Deg2Rad);
     System.out.println("Exit from MathSin. Res = "+res);
   }
}
class MathCos extends Thread {
     public double deg;
     public double res;
```

# A JAVA PROGRAM WITH MULTIPLE THREADS

```
    public MathCos(int degree) {
        deg = degree;
    }
    public void run() {
        System.out.println("Executing cos of "+deg);
        double Deg2Rad = Math.toRadians(deg);
        res = Math.cos(Deg2Rad);
       System.out.println("Exit from MathCos. Res = "+res);
    }
}
class MathTan extends Thread {
    public double deg;
    public double res;
    public MathTan(int degree) {
        deg = degree;
    }
    public void run() {
        System.out.println("Executing tan of "+deg);
        double Deg2Rad = Math.toRadians(deg);
        res = Math.tan(Deg2Rad);
        System.out.println("Exit from MathTan. Res = "+res);
    }
}
```

# A JAVA PROGRAM WITH MULTIPLE THREADS

```
class MathThreads {
      public static void main(String args[]) {
          MathSin st = new MathSin(45);
          MathCos ct = new MathCos(60);
          MathTan tt = new MathTan(30);
          st.start();
          ct.start();
          tt.start();
       try { // wait for completion of all thread and then sum
          st.join();
          ct.join(); //wait for completion of MathCos object
          tt.join();
          double z = st.res + ct.res + tt.res;
          System.out.println("Sum of sin, cos, tan = "+z);
       }
       catch(InterruptedException IntExp) {
      }
     }
}
```

# A JAVA PROGRAM WITH MULTIPLE THREADS

***Run 1:***

```
[raj@mundroo] threads [1:111] java MathThreads
Executing sin of 45.0
Executing cos of 60.0
Executing tan of 30.0
Exit from MathSin. Res = 0.7071067811865475
Exit from MathCos. Res = 0.5000000000000001
Exit from MathTan. Res = 0.5773502691896257
Sum of sin, cos, tan = 1.7844570503761732
```

***Run 2:***

```
[raj@mundroo] threads [1:111] java MathThreads
Executing sin of 45.0
Executing tan of 30.0
Executing cos of 60.0
Exit from MathCos. Res = 0.5000000000000001
Exit from MathTan. Res = 0.5773502691896257
Exit from MathSin. Res = 0.7071067811865475
Sum of sin, cos, tan = 1.7844570503761732
```

***Run 3:***

```
[raj@mundroo] threads [1:111] java MathThreads
Executing cos of 60.0
Executing sin of 45.0
Executing tan of 30.0
Exit from MathCos. Res = 0.5000000000000001
Exit from MathTan. Res = 0.5773502691896257
Exit from MathSin. Res = 0.7071067811865475
Sum of sin, cos, tan = 1.7844570503761732
```

# A JAVA PROGRAM WITH MULTIPLE THREADS

- ***Observations on Results*** A close observation of the output of three different runs reveals that the execution/completion of threads does not need to be in the order of their start (i.e., invocation of start() method).
  - In Run 1, it so happened that the output of all three threads happen to be generated in the same order of their start.
  - However, this is not the case in the next two runs. Because of various operations that are performed within JVM, threads, and different applications running at different times, it is hard to predict the precise time at which context-switching occurs.
  - It is possible to enforce the order of execution to some extent by setting different priorities for threads.

# THREAD PRIORITY

- It is important for different threads to have different priorities.
- Important threads should always have higher priority than less important ones.
- For example, the collector thread just needs the lowest priority to execute, which means it will not be executed before all other threads are scheduled to run.
- It is possible to control the priority of the threads by using the Java APIs.
- The Thread.setPriority(…) method serves this purpose.
- The Thread class provides 3 constants value for the priority:

MIN_PRIORITY = 1, NORM_PRIORITY = 5, MAX_PRIORITY = 10

- The priority range of the thread should be between the minimum and the maximum number.
- The following example shows how to alter the order of the thread by changing its priority.

# Thread Priority

- In Java, each thread is assigned priority, which affects the order in which it is scheduled for running.
- The threads so far had same default priority (NORM_PRIORITY).
  - Java allows users to change priority:
    - ThreadName.setPriority(intNumber)
      - MIN_PRIORITY = 1
      - NORM_PRIORITY=5
      - MAX_PRIORITY=10

# Thread Priority Example

```
class A extends Thread
{
      public void run()
       {
          System.out.println("Thread A started");
          for(int i=1;i<=4;i++)
           {
               System.out.println("\t From ThreadA: i= "+i);
           }
            System.out.println("Exit from A");
       }
}
class B extends Thread
{
      public void run()
       {
          System.out.println("Thread B started");
          for(int j=1;j<=4;j++)
           {
               System.out.println("\t From ThreadB: j= "+j);
           }
            System.out.println("Exit from B");
       }
}
```

# Thread Priority Example

```
class C extends Thread
{
      public void run()
       {
           System.out.println("Thread C started");
           for(int k=1;k<=4;k++)
            {
                System.out.println("\t From ThreadC: k= "+k);
            }
             System.out.println("Exit from C");
       }
}
```

# Thread Priority Example

```
class ThreadPriority
{
        public static void main(String args[])
         {
                  A threadA=new A();
                  B threadB=new B();
                  C threadC=new C();
                threadC.setPriority(Thread.MAX_PRIORITY);
                threadB.setPriority(threadA.getPriority()+1);
                threadA.setPriority(Thread.MIN_PRIORITY);
                System.out.println("Started Thread A");
                threadA.start();
                System.out.println("Started Thread B");
                threadB.start();
                System.out.println("Started Thread C");
                 threadC.start();
                 System.out.println("End of main thread");
         }
}
```

# Thread Priority Example

**This example creates 3 threads, and adjusts the priority of each thread.**
**The possible output of the example is shown as follows:**

```
Started Thread A
Started Thread B
Started Thread C
Thread B started
End of main thread
Thread C started
From ThreadC: k= 1
From ThreadC: k= 2
From ThreadC: k= 3
From ThreadC: k= 4
Exit from C
From ThreadB: j= 1
From ThreadB: j= 2
From ThreadB: j= 3
From ThreadB: j= 4
Exit from B
Thread A started
From ThreadA: i= 1
From ThreadA: i= 2
From ThreadA: i= 3
From ThreadA: i= 4
Exit from A
```

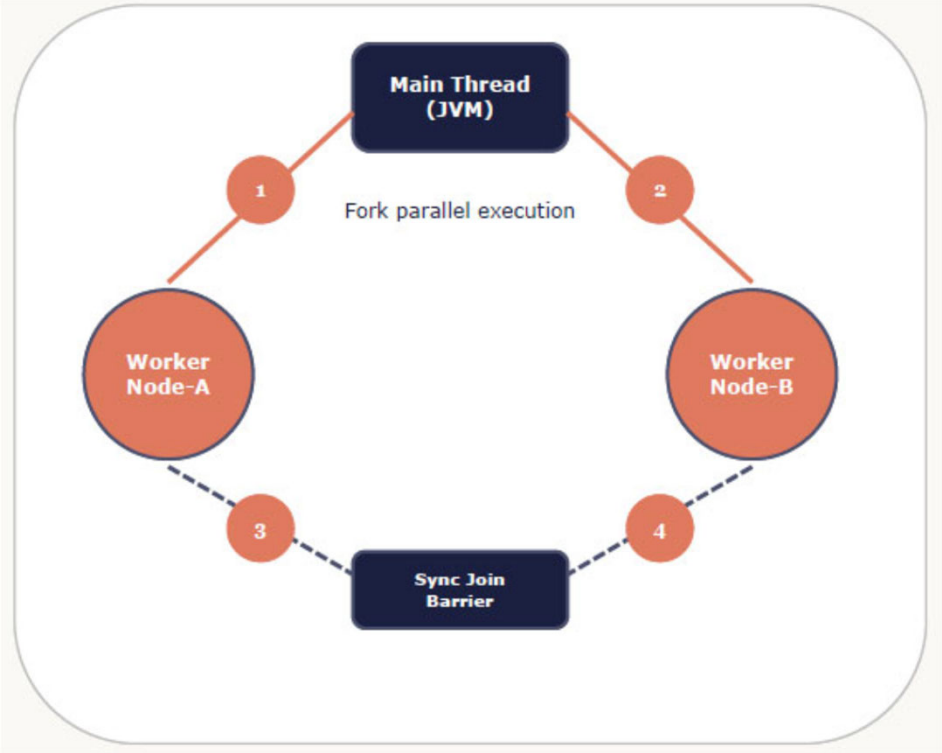
**The highest priority thread C is scheduled/executed first and the thread A which has the lowest priority is scheduled last.**

# PROGRAM#1

## Threads perform tasks concurrently

### Simple Program: Concurrent Execution in a Distributed System

This program demonstrates the **Concurrent Execution** characteristic of a distributed system. Two independent nodes, **Node-A** and **Node-B**, perform their tasks simultaneously using separate threads. The main program waits for both threads to complete before displaying a final message.

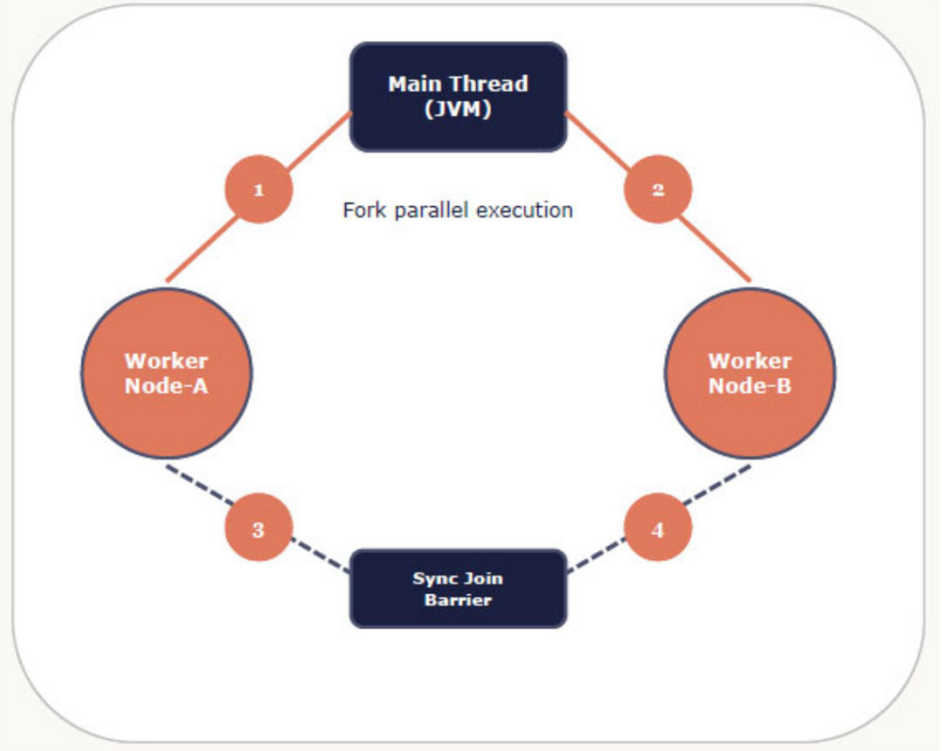


# PROGRAM#1

## Threads perform tasks concurrently

### FUNCTION OF THE PROGRAM

The program simulates two independent nodes in a distributed system executing tasks **concurrently**. By using separate threads, Node-A and Node-B can perform their work at the same time rather than sequentially. The join() method ensures that the main program waits until both nodes finish before printing:
All nodes have completed their work.
This demonstrates the concept of **parallel/concurrent execution**, a key characteristic of distributed systems.



```
public class ConcurrentExecutionDemo {
public static void main(String[] args) {
// Node-A task
      Thread nodeA = new Thread(() -> {
        for (int i = 1; i <= 3; i++) {
        System.out.println("Node-A processing task " + i);
      }
      });
// Node-B task
     Thread nodeB = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
            System.out.println("Node-B processing task " + i);
     }
     });
    // Start both nodes concurrently
        nodeA.start();
        nodeB.start();
   // Wait for both nodes to finish
  try {
        nodeA.join();
        nodeB.join();
  }
```

## STATEMENT-BY-STATEMENT EXPLANATION

**Class Declaration**

public class ConcurrentExecutionDemo {

Defines a class named ConcurrentExecutionDemo.

The program starts execution from this class.

**Main Method**

public static void main(String[] args) {

The entry point of the program.

Execution begins here.

**Create Thread for Node-A**

Thread nodeA = new Thread(() -> {

Creates a new thread representing **Node-A**.

The code inside the braces is the task executed by Node-A.

**Node-A Tasks**

1 for (int i = 1; i <= 3; i++) {

2 System.out.println("Node-A processing task " + i);

}

Simulates Node-A performing three tasks.

Prints a message for each task.

```
catch (InterruptedException e) {
      e.printStackTrace();
  }
    // Final message
      System.out.println("All nodes have completed their
work.");
  }
  }
```

**STATEMENT-BY-STATEMENT EXPLANATION**

**Create Thread for Node-B**

1 Thread nodeB = new Thread(() -> {

Creates a second thread representing **Node-B**.

This thread runs independently of Node-A.

**Node-B Tasks**

1 for (int i = 1; i <= 3; i++) {

2 System.out.println("Node-B processing task " + i);

3 }

Simulates Node-B performing three tasks.

Prints a message for each task.

**Start Node-A**

nodeA.start();

Starts execution of Node-A's thread.

**Start Node-B**

nodeB.start();

Starts execution of Node-B's thread.

Both threads now run concurrently.

## STATEMENT-BY-STATEMENT EXPLANATION

**Wait for Node-A**

```
nodeA.join();
```

Causes the main thread to wait until Node-A finishes.

**Wait for Node-B**

```
nodeB.join();
```

Causes the main thread to wait until Node-B finishes.

**Exception Handling**

```
1 catch (InterruptedException e) {
2 e.printStackTrace();
3 }
```

Handles the case where a waiting thread is interrupted.

**Final Completion Message**

```
System.out.println("All nodes have completed their work.");
```

Executes only after both nodes finish their tasks.

## Sample Output

Because threads execute concurrently, the order may vary each time the program runs.

One possible output is:

```
 1 Node-A processing task 1
2 Node-B processing task 1
3 Node-A processing task 2
4 Node-B processing task 2
5 Node-A processing task 3
6 Node-B processing task 3
7 All nodes have completed their work.
```

Another possible output is:

```
1 Node-B processing task 1
2 Node-B processing task 2
3 Node-A processing task 1
4 Node-A processing task 2
5 Node-B processing task 3
6 Node-A processing task 3
7 All nodes have completed their work.
```

# PROGRAM#2

## Threads perform multiple clients on one machine

Write a program in java to do the following task:
If you want to simulate **multiple clients on one machine** and have them **send requests simultaneously** to a server in Java.

- you can create multiple client threads.
- Each thread acts as an independent client with its own socket connection.

Java for implementing multiple clients on one machine

```
import java.io.*; // Imports classes for input/output operations
import java.net.*; // Imports classes for network communication
public class MultiClient {
// Main method: execution starts here
public static void main(String[] args) {
// Number of client threads to create
int numberOfClients = 10;
// Loop to create multiple clients
for (int i = 1; i <= numberOfClients; i++) {
// Store client ID as a final variable so it can be used inside the
lambda expression
final int clientId = i;
// Create a new thread representing a client
Thread clientThread = new Thread(() -> {
try {
// Connect to the server running on localhost at port 5000
Socket socket = new Socket("localhost", 5000);
// Create an output stream for sending messages to the server
// 'true' enables automatic flushing after println()
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
// Create an input stream for receiving messages from the server
BufferedReader in = new BufferedReader(
new InputStreamReader(socket.getInputStream()));
// Create a unique message for this client
String msg = "Hello from Client " + clientId;
```

## Client Execution Flow

1. Start Program
2. Create 10 client threads
3. Each thread connects to the server on port 5000
4. Each client sends: "Hello from Client X"
5. The server receives the message
6. The server sends a response back
7. The client displays the response
8. The socket is closed
9. All clients run concurrently using separate threads

```
// Send the message to the server
out.println(msg);
// Read the server's reply
String response = in.readLine();
// Display the server's response on the client's console
System.out.println( "Client " + clientId + " received: " + response);
// Close the socket connection after communication is complete
socket.close();
} catch (Exception e) {
// Print error details if an exception occurs
e.printStackTrace();
}
});
// Start the client thread
// This allows multiple clients to run concurrently
clientThread.start();
}
}
}
```

Java for implementing server receive requests from multiple clients

```
import java.io.*; // Imports classes for input/output operations
import java.net.*; // Imports classes for networking operations
// Main server class
public class MultiThreadServer {
// Main method: execution starts here
public static void main(String[] args) {
try {
// Create a ServerSocket that listens for client connections on port 5000
ServerSocket serverSocket = new ServerSocket(5000);
// Display a message indicating that the server has started
System.out.println("Server started...");
// Infinite loop to keep the server running and accepting clients
while (true) {
// Wait for a client to connect
// accept() blocks until a client establishes a connection
Socket clientSocket = serverSocket.accept();
// Print a message when a new client successfully connects
System.out.println("New client connected");
// Create a ClientHandler object to manage communication with this client
ClientHandler handler = new ClientHandler(clientSocket);
```

```
// Create and start a new thread for the connected client
// This allows multiple clients to be handled simultaneously
new Thread(handler).start();
}
} catch (IOException e) {
// Print error details if a networking exception occurs
e.printStackTrace();
}
}
}
// Class responsible for handling communication with a single client
class ClientHandler implements Runnable {
// Stores the socket associated with the connected client
private Socket socket;
// Constructor receives the client's socket
public ClientHandler(Socket socket) {
// Assign the received socket to the instance variable
this.socket = socket;
}
// Method executed automatically when the thread starts
public void run() {
try {
// Create an input stream to receive data from the client
BufferedReader in = new BufferedReader(
new InputStreamReader(socket.getInputStream()));
```

## Server Execution Flow

1. Start the server.
2. Open port 5000 using ServerSocket.
3. Wait for client connections.
4. When a client connects:
   - Create a ClientHandler object.
   - Create a new thread.
   - Start the thread.
5. The thread:
   - Receives a message from the client.
   - Displays the message.
   - Sends a response back.
   - Closes the connection.
6. The server continues waiting for additional clients.

```
// Create an output stream to send data to the client
// 'true' enables automatic flushing after println()
PrintWriter out = new PrintWriter(
socket.getOutputStream(), true);
// Read one line of text sent by the client
String message = in.readLine();
// Display the received message on the server console
System.out.println("Received: " + message);
// Send a response back to the client
out.println("Server received: " + message);
// Close the client's socket after communication is finished
socket.close();
} catch (IOException e) {
// Print error details if an exception occurs
e.printStackTrace();
}
}
}
```

## Expected Output

### Server

1 Server started...
2 New client connected
3 Received: Hello from Client 1
4 Received: Hello from Client 2
5 Received: Hello from Client 3
6...

### Clients

Plain Text
1 Client 1 received: Server received:
Hello from Client 1
2 Client 2 received: Server received:
Hello from Client 2
3 Client 3 received: Server received:
Hello from Client 3
4...

# END

End

End

End